

Sujets de mini-mémoires

Fiches de démarrage – 10 sujets au choix

Jean Delpech | M1 Data Science

#	Sujet	Parcours	Code	Théorie
1	Tables de hachage	Les deux	Moyenne	Faible
2	Index BDD et B-tree	Data Eng.	Moyenne	Moyenne
3	Tries et auto-complétion	Les deux	Faible	Faible
4	Graphes et recommandation	IA/Data	Moyenne	Moyenne
5	Dijkstra, A* et géospatial	Les deux	Moyenne	Faible
6	Tri avancé et Map/Reduce	Data Eng.	Faible	Moyenne
7	DP et séries temporelles	IA/Data	Moyenne	Élevée
8	Moteur de recherche textuelle	Les deux	Moyenne	Moyenne
9	Bases vectorielles	IA/Data	Élevée	Élevée
10	Gradient boosting	IA/Data	Faible	Élevée

Consignes générales

Format du mémoire

- Un mémoire écrit d'une douzaine de pages (hors annexes et bibliographie) rédigé en binôme (cf. les specs plus précises dans le document « guide_memoire.md » ci-joint).
- Une présentation orale de 15 minutes devant toute la classe (qui se livrera à une co-évaluation), suivie de 5 minutes de questions.
- Une démo : application Streamlit ou notebook Jupyter selon ce qui est le plus adapté à votre sujet.
- Le mémoire doit comporter : introduction et problématique, présentation théorique des algorithmes, implémentation commentée, application et résultats, analyse des limites, bibliographie (encore une fois, cf. les specs plus précises dans le document « guide_memoire.md » ci-joint).

Les sujets présentés

- Les sujets présentés le sont à titre indicatifs : ils ont vocation à vous faire découvrir un champ théorique et un champ d'application de structures de données que l'on verra en cours et dans quelle mesure elles sont pertinentes et utilisées en data science/data engineering.
- Les sujets sont présentés de manière structurée afin de vous permettre de délimiter le champ théorique et vous aider à formuler une problématique, vous aider à démarrer, et à vous guider pour aller plus loin (implémentation, application à un dataset, etc.). C'est un modèle.
- Vous n'êtes donc pas obligés de suivre ce guide à la lettre, et vous avez le droit de proposer vos propres sujets si vous êtes capables de les formaliser de la même manière. Par contre vous devez me les soumettre avant la séance du vendredi 29 mai pour validation.

Livrables attendus

- Mémoire écrit (donc au format .PDF) déposé la veille de la présentation.
- La grille de co-évaluation de vos camarades.
- Code source du mémoire et de la démo (dépôt Git ou archive).
- Support de présentation (slides) utilisé pendant les 15 minutes.

Sur les datasets

Pour la démo, **vous générerez un dataset synthétique** adapté à votre sujet plutôt que d'utiliser un dataset réel. L'objectif est que votre dataset mette en valeur les avantages et les limites de l'algorithme que vous présentez, pas de traiter un jeu de données brut. Bien sûr, une fois cela fait, je ne vous interdit pas de tester votre implémentation sur un jeu de données réel si vous en disposez. Néanmoins, pour certains sujets, il est possible que partir d'un jeu de données réel (p. ex. corpus de textes, réseaux routier...) ne pose pas moins de difficulté, ce sera à vous de juger.

- Le dataset doit être générable de façon reproductible avec une graine aléatoire fixée (random_state=42). Attention, pour certains sujets il faut penser à créer une relation entre les variables (p. ex. si vous créez des fausses données immobilières, il faudra par exemple un lien entre surface et prix – et d'autres variables bien sûr – que votre régression devra retrouver, n'oubliez pas non plus de rajouter du bruit.)
- Il doit être suffisamment grand pour que les différences de performance soient mesurables.
- Il doit contenir des cas favorables et des cas limites pour l'algorithme étudié.

Vous expliquerez comment vous avez choisi de générer le dataset dans une annexe de votre mémoire.

Sur l'utilisation de l'IA

Vous êtes autorisés à utiliser l'IA (Claude, ChatGPT, Gemini...) pour vos recherches, pour mieux comprendre un concept, pour déboguer du code. **À une condition : vous devrez être capables de ré-expliquer par vous-mêmes tout ce que vous présentez.**

- Toute utilisation de l'IA doit être mentionnée dans le mémoire (section 'Outils utilisés').
- Les questions posées à l'oral serviront à vérifier votre compréhension réelle (vous devrez réexpliquer un raisonnement, une preuve...). Copier/coller des réponses d'IA ne vous apportera rien.
- Un étudiant qui cite du code généré par IA sans en comprendre le fonctionnement sera en difficulté à l'oral.
- Le « guide_memoire.md » vous donne quelques indices sur la manière d'utiliser l'IA « intelligemment »

Évaluation :

- Mémoire écrit : clarté des explications (théorie), respect du plan, rigueur de l'implémentation, qualité de l'analyse
- Présentation orale : clarté de l'exposé, respect du temps, aisance, pertinence des exemples, qualité de la démo
- Questions : compréhension réelle des algorithmes, capacité à expliquer les choix. Attention, cette partie permettra d'individualiser les notes : quelle que soit la manière dont vous vous êtes partagé le travail n'importe lequel d'entre vous devra être capable de répondre sur l'ensemble du sujet abordé : préparez-vous bien à deux, testez-vous entre vous.
- Je ne vous demande pas d'écrire une thèse. L'objectif de l'exercice est de vous confronter à un sujet/problème inconnu, à rechercher des infos, comprendre quelles bases théoriques utiliser, les assimiler, et proposer une solution. Ce sera votre quotidien de développeur. Un sujet bien délimité, exposé clairement sera nettement préférable à une recherche d'exhaustivité où vous vous perdrez et perdrez votre auditoire.
- Bonus : bien sûr je n'interdis pas – si vous êtes passionné-e-s par le sujet – d'aller aussi loin que vous voulez, plus vous irez loin et plus vous aurez des points bonus (à vos risques et périls, cf. point précédent). Par exemple si vous désirez tester votre implémentation sur un dataset réel (et que vous en trouvez un) et non seulement sur un dataset artificiel, ce sera du bonus aussi. Mais assurez le principal avant de vous lancer là-dedans.
- Soyez stratégique : si vous avez choisi un sujet au-delà de vos forces, réduisez le scope, compensez en approfondissant la partie théorique ou bibliographique, mettez l'accent sur l'analyse, bref

garantisiez ce que vous pouvez assurer. Vous n'aurez peut-être pas tout les points, mais ce sera toujours mieux qu'un mémoire inachevé, avec aucune partie terminée car vous aurez été découragé-e.

- Les sujets peuvent sembler très ambitieux (ils le sont). C'est fait exprès : c'est pour vous pousser à aller le plus loin possible. Donc allez le plus loin possible, ne vous mettez pas la pression pour « tout » faire (cf. point précédent). On évaluera le point d'arrivée après coup.
- Certains sujets sont plus difficiles que d'autres. Évidemment j'attendrais une analyse théorique impeccable et vraiment complète pour les sujets les plus faciles.

Les tables de hachage

Parcours : Data Engineering (accessible aux deux parcours) | Difficulté code : Moyenne | Difficulté théorie : Faible

Théorie mobilisée

Une table de hachage est une structure de données qui associe des clés à des valeurs avec un accès moyen en $O(1)$. Son mécanisme central est la fonction de hachage, qui transforme une clé en un indice dans un tableau. Lorsque deux clés produisent le même indice – une collision – il faut une stratégie de résolution. Les deux grandes familles sont le chaînage séparé (chaque case pointe vers une liste de paires) et l'open addressing (on cherche la prochaine case libre dans le tableau). La performance dépend du facteur de charge $\alpha = n/m$, où n est le nombre d'éléments stockés et m la taille du tableau interne (nombre de cases allouées). Ce rapport mesure le taux de remplissage du tableau : quand α dépasse un seuil (0.75 pour dict Python), le tableau est redimensionné. Ce mécanisme explique pourquoi les opérations sur dict Python sont en $O(1)$ amorti et non $O(1)$ garanti.

Angle du mémoire

Pourquoi dict Python est une des structures de données les plus utilisées au monde, et ce qui se passe vraiment quand on écrit `d[key] = value`. Le mémoire reconstruit une table de hachage from scratch, compare les deux stratégies de résolution de collision, et mesure expérimentalement l'impact du facteur de charge sur les performances.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Lire la documentation Python sur `__hash__` et `__eq__` pour comprendre comment ça se passe « sous le capot » en Python
2. Implémenter une HashTable minimale avec chaînage (insert, get, delete) – sans se soucier du resize pour commencer
3. Ajouter le redimensionnement dynamique et observer l'impact sur les performances
4. Implémenter la variante open addressing avec linear probing et le marqueur tombstone pour la suppression
5. Chercher 'Python dict implementation CPython' pour voir comment Python le fait en vrai
6. Chercher 'hash table collision resolution comparison' pour trouver des benchmarks publiés

Application data science

Pipeline de déduplication d'un référentiel client : générer 100 000 noms avec 15% de doublons exacts (même chaîne) et mesurer le temps de déduplication avec une HashTable maison vs set Python vs `pandas.duplicated()`. Montrer l'effet du facteur de charge sur les performances de la HashTable maison.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Application Streamlit avec un curseur de facteur de charge qui affiche en temps réel la courbe de performance et le taux de collision. Ou notebook avec visualisation de la distribution des clés dans le tableau selon différentes fonctions de hachage.

Exemples ; de ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi $O(1)$ amorti et non $O(1)$ garanti ? Quelle opération coûte $O(n)$ et quand survient-elle ?
- Pourquoi on ne peut pas hacher une liste Python mais on peut hacher un tuple ?
- Dans quel cas un index Hash PostgreSQL est-il préférable à un index B-tree, et pourquoi ?
- Qu'est-ce que le clustering primaire dans l'open addressing et comment le limiter ?

Les arbres de recherche et les index de base de données

Parcours : Data Engineering (prioritaire) | Difficulté code : Moyenne | Difficulté théorie : Moyenne

Théorie mobilisée

Un arbre binaire de recherche (BST) est une structure récursive où chaque nœud stocke une valeur, un sous-arbre gauche (valeurs inférieures) et un sous-arbre droit (valeurs supérieures ou égales). La hauteur h de l'arbre détermine la complexité des opérations : $O(h)$ pour la recherche, l'insertion et la suppression. Sur des données aléatoires, $h \approx \log n$, mais sur des données triées, l'arbre dégénère en liste chaînée ($h = n$). L'équilibrage AVL résout ce problème en maintenant à chaque nœud un facteur d'équilibre $| \text{hauteur(gauche)} - \text{hauteur(droite)} | \leq 1$, via ce que l'on appelle des rotations. Une rotation est une opération locale qui va permettre le rééquilibrage sans modifier l'ordre des éléments, car on ne peut pas réorganiser les nœuds n'importe comment si on veut pour voir les parcourir encore dans l'ordre (ce qui fait l'intérêt de cette structure de données – BST – pour les bases de données). Le B-tree généralise le BST à m enfants par nœud, optimisé pour les accès disque où un nœud = une page disque. Le B+ tree (variante standard dans les SGBD) stocke les données uniquement dans les feuilles chaînées entre elles. PostgreSQL utilise des index B-tree pour quasiment tous les index par défaut, et des index Hash pour les égalités pures.

Angle du mémoire

Quand on écrit `CREATE INDEX` en SQL, qu'est-ce qui se passe réellement sur le disque ? Le mémoire part du BST simple, démontre sa dégradation sur des données pathologiques, explique pourquoi les SGBD ont besoin du B-tree, et relie tout à un vrai plan d'exécution PostgreSQL lu avec `EXPLAIN ANALYZE`.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Implémenter un BST naïf (insert, search, in-order traversal) et visualiser sa structure
2. Tester la dégradation : insérer 1000 éléments triés et mesurer la hauteur obtenue
3. Comprendre intuitivement les 4 rotations AVL avec des schémas avant le code
4. Chercher 'B-tree vs B+ tree database' pour comprendre la différence et pourquoi B+ tree est standard
5. Installer PostgreSQL localement (ou utiliser un service cloud gratuit), créer une table et comparer les plans `EXPLAIN ANALYZE` avec et sans index
6. Chercher 'PostgreSQL index types when to use' pour le guide officiel sur le choix d'index

Application data science

Générer un dataset de logs de transactions (1 million de lignes : id, date, montant, client_id, statut) et montrer empiriquement l'impact de la création d'un index B-tree sur les temps de requête pour `ORDER BY date`, `BETWEEN montant`, `= client_id`. Inclure un cas de requête qui n'utilise pas l'index pour montrer les limites.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Notebook avec visualisation du BST construit pas à pas et benchmark PostgreSQL avec et sans index sur les différents types de requêtes. Un graphique temps de requête / nombre de lignes pour les deux cas.

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi un index accélère les lectures mais ralentit les écritures ?
- Qu'est-ce qu'un Seq Scan, un Index Scan et un Bitmap Scan dans un plan PostgreSQL ?
- Pourquoi le B-tree a-t-il un ordre m élevé (typiquement 100-200) en base de données mais pas en mémoire vive ?
- Dans quel cas un index sur (colA, colB) n'est-il pas utilisé pour une requête sur colB seul ?

Les Tries et la recherche de chaînes de caractères

Parcours : Les deux parcours | Difficulté code : Faible | Difficulté théorie : Faible

Théorie mobilisée

Un Trie (de l'anglais retrieval) est un arbre où chaque chemin de la racine à un nœud terminal représente un mot ou une chaîne. Contrairement à un BST qui compare des valeurs entières, chaque arête d'un Trie représente un caractère, et chaque nœud représente un préfixe. L'insertion et la recherche s'effectuent en $O(L)$ où L est la longueur de la chaîne, indépendamment du nombre de mots stockés. Le Radix Tree (ou Patricia Trie) est une variante compressée qui fusionne les nœuds à un seul enfant pour économiser la mémoire. La distance de Levenshtein complète naturellement le Trie : le Trie propose des candidats par préfixe, Levenshtein les classe par distance d'édition pour la tolérance aux fautes.

Angle du mémoire

Quand on tape dans un moteur de recherche et que les suggestions apparaissent en quelques millisecondes, quelle structure de données rend ça possible à grande échelle ? Le mémoire construit un Trie from scratch, compare ses performances avec d'autres approches, et l'applique à un vrai moteur d'auto-complétion avec tolérance aux fautes.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Construire un Trie à la main sur 10 mots – dessiner l'arbre nœud par nœud
2. Implémenter insert, search et starts_with
3. Mesurer et comparer : recherche de préfixe Trie vs liste triée avec bisect vs dictionnaire Python
4. Chercher 'Radix tree vs Trie memory comparison' pour comprendre la variante compressée
5. Ajouter la distance de Levenshtein pour classer les suggestions par distance au terme saisi
6. Chercher 'autocomplete trie python tutorial' pour voir des exemples publiés

Application data science

Générer un dictionnaire de 50 000 noms de produits e-commerce (catégorie, marque, modèle) et construire un moteur d'auto-complétion. Tester avec des saisies partielles et des fautes de frappe. Benchmark la recherche de préfixe selon les trois approches.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Application Streamlit avec un champ de saisie qui affiche les suggestions en temps réel, le temps de réponse en millisecondes, et un toggle pour activer/désactiver la tolérance aux fautes (Levenshtein).

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi la recherche dans un Trie est en $O(L)$ indépendamment du nombre de mots stockés ?
- Dans quel cas un Trie est moins efficace qu'une HashTable pour la recherche d'un mot exact ?
- Qu'est-ce qu'un nœud terminal dans un Trie et pourquoi est-il nécessaire ?
- Pourquoi le Radix Tree économise-t-il de la mémoire par rapport au Trie standard ?

Graphes et systèmes de recommandation

Parcours : IA/Data (prioritaire) | Difficulté code : Moyenne | Difficulté théorie : Moyenne

Théorie mobilisée

Un graphe est une structure composée de sommets (nœuds) et d'arêtes (liens). Il peut être orienté ou non, pondéré ou non. Les deux principales représentations sont la matrice d'adjacence ($O(V^2)$ en espace, accès $O(1)$) et la liste d'adjacence ($O(V+E)$ en espace, accès $O(\deg)$). BFS (Breadth-First Search) explore le graphe par niveaux via une file FIFO et trouve les plus courts chemins en nombre d'arêtes. DFS (Depth-First Search) explore en profondeur via une pile LIFO ou la récursion. Un graphe bipartite est un graphe dont les sommets se divisent en deux ensembles disjoints – idéal pour modéliser des relations utilisateurs $\times$ items. PageRank est un algorithme de marche aléatoire qui attribue un score d'importance à chaque nœud selon le nombre et la qualité des liens entrants. Le Random Walk sur un graphe bipartite génère des recommandations en simulant les déplacements d'un utilisateur.

Angle du mémoire

Comment Netflix ou Spotify savent-ils ce qu'on veut regarder ou écouter ensuite ? Le mémoire modélise le problème de recommandation comme un graphe bipartite utilisateurs $\times$ items et explore les algorithmes de parcours pour produire des recommandations personnalisées, évaluées avec des métriques standard.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Implémenter BFS et DFS from scratch sur une liste d'adjacence
2. Construire un graphe bipartite utilisateurs $\times$ films à la main sur un exemple de 5 utilisateurs et 8 films
3. Chercher 'collaborative filtering graph based recommendation' pour l'angle data science
4. Implémenter PageRank from scratch (version itérative) et l'appliquer au graphe bipartite
5. Chercher 'random walk recommendation system' pour le deuxième algorithme de recommandation
6. Chercher 'precision at k recall at k recommendation metrics' pour les métriques d'évaluation

Application data science

Générer un dataset de 500 utilisateurs $\times$ 200 films avec des notes (1 à 5) – s'inspirer du format MovieLens mais générer les données synthétiquement. Construire deux systèmes de recommandation (PageRank et Random Walk) et évaluer avec précision@5 et rappel@5 sur 20% des données laissées de côté.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Application Streamlit : saisir un identifiant utilisateur, afficher ses films déjà notés, et les recommandations générées par chaque algorithme avec leur score. Un onglet pour comparer les deux approches.

Ce que vous devrez être capables d'expliquer à l'oral

- La différence entre le collaborative filtering (basé sur les utilisateurs) et le content-based filtering (basé sur les attributs des items) ?
- Pourquoi PageRank converge-t-il et que représente le facteur d'amortissement (damping factor) ?
- Qu'est-ce que la précision@K et le rappel@K – comment les calculer concrètement ?
- En quoi un graphe bipartite est-il adapté à la modélisation utilisateurs × items ?

Dijkstra, A* et la planification d'itinéraires géospatiaux

Parcours : Les deux parcours | Difficulté code : Moyenne | Difficulté théorie : Faible

Théorie mobilisée

Dijkstra est un algorithme de plus court chemin dans un graphe orienté pondéré à poids positifs. Il maintient un ensemble de distances provisoires et explore les nœuds dans l'ordre croissant de leur distance via un min-heap (file de priorité). Sa complexité est $O((V+E) \log V)$ avec un heap. La distance Haversine calcule la distance en ligne droite entre deux points à la surface de la Terre en tenant compte de sa courbure – contrairement à la distance euclidienne sur les coordonnées (lat, lon) qui introduit une erreur croissante avec la latitude. A* est une extension de Dijkstra guidée par une heuristique $h(n)$ qui estime la distance restante jusqu'à la destination – la distance Haversine est une heuristique admissible naturelle pour les problèmes géospatiaux. A* explore moins de nœuds que Dijkstra en pratique, avec la même garantie d'optimalité si h est admissible.

Angle du mémoire

Quand Google Maps calcule le plus court chemin en quelques secondes sur un réseau de millions de routes, il ne teste pas tous les itinéraires possibles. Le mémoire explique comment Dijkstra exploite un Heap pour explorer intelligemment le graphe routier, et pourquoi A* fait mieux grâce à une heuristique géographique.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Implémenter Dijkstra from scratch avec heapq et le tester sur un graphe de 10 villes
2. Implémenter la distance Haversine et vérifier contre geopy.distance
3. Chercher 'OSMnx tutorial' pour charger un réseau routier réel depuis OpenStreetMap
4. Implémenter A* en ajoutant l'heuristique Haversine à Dijkstra – comparer le nombre de nœuds explorés
5. Chercher 'A* admissible heuristic' pour comprendre la condition de garantie d'optimalité
6. Visualiser le résultat avec folium (carte interactive HTML)

Application data science

Charger le réseau routier d'une ville française avec OSMnx. Générer 10 points de livraison aléatoires dans la ville. Calculer les plus courts chemins entre l'entrepôt et chaque point de livraison avec Dijkstra et A*. Comparer le nombre de nœuds explorés et les temps de calcul pour les deux algorithmes.

Dataset : générer un dataset synthétique à partir de données réelles adaptées à votre démonstration (cf. consignes en début de document). Si un réseau routier est trop compliqué à gérer, chercher d'autres données en réseau (villes plus petites par exemple), enfin créer un graphe complexe totalement artificiel en dernier recours.

Démo suggérée

Application Streamlit avec une carte folium intégrée : cliquer deux points sur la carte, afficher le chemin calculé par Dijkstra et A*, avec le nombre de nœuds explorés par chaque algorithme affiché à côté.

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi Dijkstra ne fonctionne pas avec des poids négatifs ?
- Qu'est-ce qui rend une heuristique admissible, et pourquoi la distance Haversine l'est-elle ?
- Pourquoi la distance euclidienne sur (lat, lon) est-elle incorrecte, et quelle est l'erreur à la latitude de Paris ?
- Que se passe-t-il si on retire le Heap de Dijkstra et qu'on utilise une liste triée à la place ?

Tri avancé et le paradigme Map/Reduce

Parcours : Data Engineering (prioritaire) | Difficulté code : Faible | Difficulté théorie : Moyenne

Théorie mobilisée

Mergesort est un algorithme de tri par division récursive : diviser le tableau en deux moitiés, trier chacune, fusionner. Sa complexité est $O(n \log n)$ dans tous les cas, et il est stable (les éléments égaux conservent leur ordre relatif). Quicksort partitionne le tableau autour d'un pivot et trie récursivement chaque partition. Sa complexité moyenne est $O(n \log n)$ mais peut dégénérer en $O(n^2)$ sur des données pathologiques. Timsort, l'algorithme derrière `list.sort()` et `pandas.sort_values()`, hybride Mergesort et Insertion Sort en détectant les séquences déjà ordonnées. Le tri externe résout le problème du tri de données trop volumineuses pour la RAM : trier des blocs en mémoire, les écrire sur disque, puis les fusionner avec un Heap. Le paradigme Map/Reduce suit exactement la structure de Mergesort distribué : chaque nœud trie sa partition (Map), les partitions triées sont fusionnées entre nœuds (Reduce).

Angle du mémoire

Quand Spark trie un DataFrame de 100 Go sur un cluster, il fait fondamentalement la même chose que Mergesort – mais distribué sur des dizaines de machines. Le mémoire part des implémentations from scratch pour remonter jusqu'à l'architecture Map/Reduce et au tri externe sur des fichiers trop grands pour la RAM.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Implémenter Mergesort et Quicksort from scratch et les comparer empiriquement sur différentes distributions
2. Chercher 'Timsort algorithm explained' pour comprendre pourquoi Python l'utilise
3. Implémenter le tri externe : lire un fichier par chunks, trier chaque chunk, fusionner avec `heapq.merge`
4. Chercher 'MapReduce sort algorithm' pour voir le lien avec Hadoop/Spark
5. Tester Quicksort sur des données triées pour observer la dégradation $O(n^2)$
6. Chercher 'stable sort vs unstable sort python' pour comprendre l'enjeu du tri multi-critères

Application data science

Générer un fichier CSV de 500 Mo (trop grand pour pandas en RAM en une fois) contenant des logs serveur : `timestamp`, `ip_client`, `code_http`, `taille_bytes`. Implémenter le tri externe chunk par chunk et comparer le résultat et le temps avec `pandas.sort_values` sur un fichier qui tient en RAM.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Notebook avec trois parties : (1) benchmark Mergesort vs Quicksort vs Timsort sur 6 distributions différentes (aléatoire, trié, trié inverse, presque trié, beaucoup de doublons, alternant), (2) démonstration de la dégradation Quicksort, (3) démonstration du tri externe.

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi Mergesort est stable et Quicksort ne l'est pas – et ce que ça change pour un tri multi-critères ?
- Dans quel cas Quicksort est-il plus rapide que Mergesort en pratique, malgré la même complexité asymptotique ?
- Comment le tri externe utilise-t-il un Heap pour fusionner k fichiers triés en une seule passe ?
- Pourquoi Timsort est-il particulièrement efficace sur des données réelles (partiellement ordonnées) ?

Programmation dynamique (matching de texte, similarité de séries temporelles)

Parcours : IA/Data (prioritaire) | Difficulté code : Moyenne | Difficulté théorie : Élevée

Théorie mobilisée

La programmation dynamique (DP) résout des problèmes d'optimisation en exploitant deux propriétés : la sous-structure optimale (la solution optimale se construit à partir de solutions optimales de sous-problèmes) et le chevauchement des sous-problèmes (les mêmes sous-problèmes sont rencontrés plusieurs fois). En mémorisant ou en tabulant les résultats, on transforme une complexité exponentielle en complexité polynomiale. La distance de Levenshtein mesure le nombre minimal d'opérations élémentaires (insertion, suppression, substitution) pour transformer une chaîne en une autre. Dynamic Time Warping (DTW) généralise ce principe aux séries temporelles numériques : il trouve l'alignement optimal entre deux séries en autorisant une déformation temporelle, c'est-à-dire qu'un point de la première série peut être aligné avec plusieurs points consécutifs de la seconde. On parle aussi de distortion ou de recalage temporel. Cela permet de comparer des séries décalées ou étirées dans le temps sans que cette différence de rythme pénalise le score de similarité, là où la corrélation de Pearson exige un alignement point à point. Sa complexité est $O(n \cdot m)$ – coûteux sur de longues séries, ce qui a motivé des variantes approximatives comme FastDTW.

Angle du mémoire

Comment Spotify détecte-t-il que deux enregistrements d'une même chanson sont similaires même si l'un est joué plus vite ? Le mémoire part de la distance de Levenshtein comme cas le plus simple de DP et monte jusqu'au DTW appliqué aux séries temporelles, en montrant le pattern commun qui relie les deux algorithmes.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Construire la matrice de Levenshtein à la main sur un exemple 5×5 et comprendre la récurrence avant le code
2. Implémenter Levenshtein bottom-up et visualiser la matrice avec une heatmap matplotlib
3. Chercher 'Dynamic Time Warping explained intuitively' (notamment des articles avec visualisations)
4. Implémenter DTW from scratch et visualiser l'alignement entre deux séries
5. Comparer DTW avec la corrélation de Pearson sur des séries décalées, chercher quand DTW est meilleur
6. Chercher 'FastDTW approximation' pour la limite de complexité et la solution industrielle

Application data science

Générer un dataset de 200 courbes de ventes mensuelles sur 24 mois pour différents produits, avec des décalages saisonniers artificiels de 1 à 3 mois entre des produits de même famille. Construire un système de clustering basé sur la similarité DTW et comparer avec un clustering basé sur la corrélation de Pearson. Montrer que DTW groupe correctement les produits décalés là où Pearson échoue.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Application Streamlit avec deux onglets : (1) Démo Levenshtein – entrer deux mots et afficher la matrice DP colorée avec le chemin optimal, (2) Démo DTW – sélectionner deux courbes et afficher leur alignement avec le score de similarité comparé à Pearson.

Ce que vous devrez être capables d'expliquer à l'oral

- Qu'est-ce que la sous-structure optimale ? donner un exemple concret avec Levenshtein
- Pourquoi le DTW est-il adapté aux séries décalées là où la corrélation de Pearson échoue ?
- Quelle est la différence entre la version top-down (mémoïsation) et bottom-up (tabulation) de la DP ?
- Dans quel cas DTW donne-t-il un résultat trompeur, quelles sont ses limites ?

Moteur de recherche textuelle : TF-IDF, BM25 et index inversé

Parcours : Les deux parcours | Difficulté code : Moyenne | Difficulté théorie : Moyenne

Théorie mobilisée

Un index inversé associe à chaque terme du corpus la liste des documents qui le contiennent (posting list), permettant de répondre à une requête sans parcourir l'intégralité du corpus. TF-IDF (Term Frequency – Inverse Document Frequency) mesure la pertinence d'un terme pour un document : TF quantifie la fréquence du terme dans le document, $IDF = \log(N/df_t)$ pénalise les termes trop communs dans le corpus. Dans la formule IDF précédente, N est le nombre total de documents dans le corpus et df_t le nombre de documents contenant le terme t au moins une fois. Plus un terme apparaît dans beaucoup de documents (df_t élevé), plus son IDF est faible, il est alors considéré comme peu discriminant. Par exemple le mot "le" apparaît dans presque tous les documents : son IDF est proche de zéro. Le mot "hachage" n'apparaît que dans quelques documents spécialisés : son IDF est élevé. Le score TF-IDF d'un document pour une requête est la somme des produits $TF \times IDF$ de chaque terme de la requête. BM25 (Best Match 25) corrige deux limites de TF-IDF : la saturation du TF (un terme répété 100 fois n'est pas $100 \times$ plus pertinent) via le paramètre k_1 , et la prise en compte de la longueur du document via le paramètre b . BM25 est le standard de facto dans Elasticsearch, Apache Lucene et PostgreSQL full-text search. Les n-grams (séquences de n caractères consécutifs) permettent une recherche approximative tolérante aux fautes de frappe via la similarité de Jaccard entre ensembles de n-grams.

Angle du mémoire

Quand on lance une recherche sur Elasticsearch ou sur la barre de recherche d'un site e-commerce et que les résultats pertinents s'affichent en millisecondes parmi des millions de documents, quel mécanisme rend cela possible ? Le mémoire construit un moteur de recherche from scratch, de l'indexation à la recherche avec scoring.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Construire un index inversé à la main sur 5 phrases courtes – dessiner la structure { terme → [doc_ids] }
2. Implémenter l'index inversé en Python et l'interrogation avec requête AND (intersection de listes)
3. Calculer TF-IDF from scratch et vérifier avec sklearn TfidfVectorizer sur les mêmes données
4. Chercher 'BM25 formula explained simply' – il existe des explications visuelles très accessibles
5. Implémenter BM25 from scratch et comparer les scores avec TF-IDF sur quelques exemples
6. Chercher 'PostgreSQL pg_trgm full text search' pour voir l'approche n-gram native

Application data science

Générer un corpus de 5000 descriptions de produits e-commerce (nom, catégorie, description de 50 à 200 mots). Construire un moteur de recherche complet et évaluer TF-IDF vs BM25 sur 30 requêtes de test avec les bons résultats connus. Calculer $precision@5$ pour les deux méthodes.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document). Vous pouvez assez facilement trouver sur Kaggle des données réelles du même type.

Démo suggérée

Application Streamlit avec un champ de recherche – afficher les 5 premiers résultats TF-IDF et BM25 côte à côte avec leurs scores, pour que l'utilisateur puisse comparer visuellement les deux méthodes sur sa requête.

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi IDF pénalise-t-il les termes très fréquents dans le corpus ? Donner un exemple concret
- Que corrige exactement BM25 par rapport à TF-IDF ? Illustrer avec un exemple de document répétant 50 fois le terme
- Comment l'algorithme à deux pointeurs permet-il d'intersecter deux posting lists triées en $O(|L1| + |L2|)$?
- Quelle est la similarité de Jaccard entre deux ensembles de n-grams, et pourquoi tolère-t-elle les fautes de frappe ?

Bases vectorielles et recherche sémantique

Parcours : IA/Data (prioritaire) | Difficulté code : Élevée | Difficulté théorie : Élevée

Théorie mobilisée

Un embedding est une représentation dense d'un objet (texte, image, entité) dans un espace vectoriel de dimension fixe, où la proximité géométrique reflète la similarité sémantique. La similarité cosinus $\cos(\theta) = (u \cdot v) / (||u|| \cdot ||v||)$ mesure l'angle entre deux vecteurs. Elle est insensible à la norme, ce qui la rend adaptée au texte. En haute dimension (768 pour BERT, 1536 pour OpenAI), la malédiction de la dimensionnalité fait que toutes les distances se concentrent, rendant les structures exactes (KD-tree, Ball Tree) inefficaces. Les méthodes Approximate Nearest Neighbor (ANN) acceptent un léger manque de précision en échange d'un gain de vitesse considérable. HNSW (Hierarchical Navigable Small World) construit un graphe de proximité multi-couches : les couches supérieures permettent une navigation rapide longue distance, la couche inférieure contient tous les nœuds avec leurs voisins proches. Une requête descend les couches en suivant les arêtes vers le voisin le plus proche : c'est un BFS guidé par la distance. Faiss et pgvector sont les deux bibliothèques de référence pour indexer et interroger des embeddings à grande échelle.

Angle du mémoire

Comment ChatGPT retrouve-t-il les passages pertinents d'un long document pour répondre à une question (Retrieval-Augmented Generation) ? Le mémoire explique ce qu'est un embedding, pourquoi la recherche exacte de voisins échoue en dimension 768, et comment HNSW résout ce problème. Il compare la recherche sémantique (embeddings) avec la recherche lexicale (BM25) pour montrer quand l'une est supérieure à l'autre.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Générer des embeddings de 20 phrases avec sentence-transformers et calculer la similarité cosinus entre toutes les paires à la main
2. Visualiser les embeddings en 2D avec UMAP pour observer la structure sémantique
3. Reproduire le benchmark de la malédiction de la dimensionnalité : mesurer la concentration des distances pour $d = 2, 10, 50, 200, 768$
4. Chercher 'HNSW algorithm explained' – il existe des visualisations animées du graphe multi-couches
5. Chercher 'Faiss getting started tutorial' et 'pgvector quickstart' pour les deux outils
6. Chercher 'RAG retrieval augmented generation' pour le cas d'usage industriel principal

Application data science

Générer un corpus de 2000 résumés d'articles sur un sujet au choix (générer avec des templates variés). Générer leurs embeddings avec sentence-transformers. Construire un index Faiss HNSW. Implémenter la recherche sémantique. Comparer avec BM25 sur 20 requêtes de test. Montrer des cas où la recherche sémantique retrouve des documents pertinents que BM25 manque (synonymes, paraphrases) et inversement.

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document), mais en fait ici il est assez facile de travailler avec des données réelles. Si vous cherchez des écrits assez normés, des articles scientifiques (abstracts) peuvent être intéressants.

Démo suggérée

Application Streamlit : entrer une requête en langage naturel, afficher les 5 résultats les plus proches sémantiquement et les 5 résultats BM25 côte à côte avec leurs scores, pour illustrer concrètement la différence entre recherche lexicale et sémantique.

Ce que vous devrez être capables d'expliquer à l'oral

- La différence fondamentale entre un vecteur TF-IDF (creux, sparse) et un embedding (dense) ?
- Pourquoi la recherche exacte de voisins (KD-tree, Ball Tree) est-elle inefficace en dimension 768 ?
- Le lien entre HNSW et BFS – comment se déroule une requête dans le graphe multi-couches ?
- Qu'est-ce que le rappel@K dans le contexte ANN, et comment le mesurer pratiquement ?

Gradient boosting

Parcours : IA/Data (prioritaire) | Difficulté code : Faible | Difficulté théorie : Élevée

Théorie mobilisée

La régression est un problème d'apprentissage supervisé où l'on cherche à prédire une valeur continue à partir de features. Un modèle de régression minimise une **fonction de perte**, typiquement l'erreur quadratique moyenne (MSE), qui mesure l'écart entre les prédictions et les valeurs réelles. La **descente de gradient** est l'algorithme d'optimisation qui ajuste les paramètres du modèle dans la direction qui réduit cette perte, pas à pas.

Le **boosting** est un paradigme d'ensemble qui construit une séquence de « modèles faibles » (weak learners), chacun corrigeant les erreurs du précédent. Un modèle faible est un modèle simple, typiquement un arbre de décision peu profond, dont les performances seules sont médiocres mais qui apporte une amélioration marginale à chaque itération. **Gradient Boosting** combine ces deux idées : chaque nouvel arbre est entraîné non pas sur les données originales, mais sur les **résidus** (erreurs) du modèle courant. En interprétant ces résidus comme le gradient négatif de la fonction de perte, on effectue une descente de gradient dans l'espace des fonctions plutôt que dans l'espace des paramètres. La prédiction finale est la somme pondérée de tous les arbres. XGBoost, LightGBM et CatBoost sont des implémentations industrielles de ce principe avec des optimisations (régularisation L1/L2, croissance d'arbre par feuilles, traitement des valeurs manquantes).

Angle du mémoire

Pourquoi XGBoost et LightGBM dominent-ils les compétitions Kaggle depuis 2015 sur les données tabulaires, là où les réseaux de neurones peinent à s'imposer ? Le mémoire reconstruit le raisonnement du gradient boosting depuis la régression simple jusqu'à l'ensemble d'arbres, montre intuitivement pourquoi corriger les erreurs successivement converge vers un bon modèle, et compare les performances sur un problème de prédiction concret.

Par rapport aux précédents, le présent sujet est plus orienté algorithme d'apprentissage que structures de données. Néanmoins il convoque quand même une structure de donnée vue en cours. Les « modèles faibles » du Gradient Boosting se rapporte à un arbre de décision, utilisé ici non pas pour indexer des données mais pour apprendre une fonction de prédiction. La profondeur de l'arbre joue le même rôle que dans un BST : plus il est profond, plus il est expressif, mais plus il risque de surapprendre.

Par où commencer – plan de recherche

Suivre ces étapes dans l'ordre pour démarrer sereinement, même avant d'avoir tout le cours.

1. Implémenter une régression linéaire from scratch (descente de gradient manuelle) et vérifier contre sklearn – s'assurer de comprendre la notion de résidu et de fonction de perte
2. Implémenter un arbre de décision de profondeur 1 (decision stump) from scratch – c'est le modèle faible de base
3. Implémenter un Gradient Boosting naïf from scratch : entraîner 10 stumps séquentiellement sur les résidus successifs et observer la réduction de l'erreur à chaque itération
4. Utiliser sklearn GradientBoostingRegressor et comparer avec l'implémentation maison
5. Chercher "gradient boosting explained step by step" – il existe des visualisations très pédagogiques de la convergence
6. Chercher "XGBoost vs LightGBM vs CatBoost comparison" pour comprendre les différences

Application data science

Générer un dataset synthétique de prédiction de prix immobiliers (surface, nombre de pièces, distance au centre, année de construction, présence d'un parking – 5 à 8 features avec des relations non linéaires et des interactions entre variables). Entraîner et comparer : régression linéaire, arbre de décision seul, Random Forest, et Gradient Boosting (sklearn + XGBoost). Évaluer avec RMSE et R^2 sur un jeu de test. Visualiser la convergence : RMSE en fonction du nombre d'arbres pour le Gradient Boosting. Montrer l'importance des features et discuter les hyperparamètres clés (learning rate, profondeur des arbres, nombre d'itérations).

Dataset : générer un dataset synthétique adapté à votre démonstration (cf. consignes en début de document).

Démo suggérée

Application Streamlit avec deux onglets : (1) Visualisation de la convergence – curseur sur le nombre d'arbres, graphique de la prédiction vs valeur réelle et courbe de RMSE en fonction des itérations ; (2) Comparaison des modèles – tableau et graphique des performances des quatre modèles sur le jeu de test, avec la feature importance du Gradient Boosting.

Ce que vous devrez être capables d'expliquer à l'oral

- Pourquoi entraîner le deuxième arbre sur les résidus du premier revient à faire une descente de gradient – expliquer intuitivement, sans la preuve formelle
- La différence entre bagging (Random Forest) et boosting. Les deux sont des méthodes d'ensemble mais avec des logiques opposées
- Pourquoi un learning rate faible avec beaucoup d'arbres donne généralement de meilleurs résultats qu'un learning rate élevé avec peu d'arbres ? Quel est le coût associé ?
- Dans quel cas le Gradient Boosting risque-t-il de surapprendre, et comment la régularisation y remédie ?